CO1 – AT1

Assignment Report

Part 2 of 2 – Tasks 7 to 12

Namma Metro Smart-Card Fare System

Procedural versus object-oriented design, safe input and output, testing, measurements, final recommendation and the Day-4 design challenge

Name	T.R Sangeet
Reg No	1921260245
Course Code	DSA0104
Course Name	OBJECT ORIENTED PROGRAMMING C++

TASK 7 - From Procedural to Object-Oriented

4 Marks | K5 Evaluate

7.1 The two versions

Procedural version - global variables and free functions (`task7_compare.cpp`, namespace `proc`):

```
namespace proc {
long long balance = 10000;            // global: any function may change it
long long minBalance = 5000;          // rule value stored separately
bool canTravel(long long f) { return balance - f >= minBalance; }
void deduct(long long f)    { if (canTravel(f)) balance -= f; }
}
```

Object-oriented version - `SmartCard` (`smartcard.hpp`), balance private:

```
// smartcard.hpp - SmartCard class (kept UNCHANGED between design v1 and v2)
#pragma once

constexpr long long MIN_BALANCE_PAISE = 5000;   // Rs 50 must remain on the card

class SmartCard {
public:
    explicit SmartCard(long long openingPaise = 0)
        : balance_{openingPaise > 0 ? openingPaise : 0} {}

    long long balance() const { return balance_; }

    bool recharge(long long paise)               // negative / zero recharge rejected
    {
        if (paise <= 0) return false;
        balance_ += paise;
        return true;
    }
    bool canTravel(long long fare) const         // rule: balance after fare >= Rs 50
    {
        return fare >= 0 && balance_ - fare >= MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare)                  // the ONLY way balance goes down
    {
        if (!canTravel(fare)) return false;
        balance_ -= fare;
        return true;
    }
private:
    long long balance_;                          // integer paise, never public
};
```

7.2 Result of running both (plus the public-member variant)

task7_compare.cpp output

```
== procedural ==
after Rs 36 fare: 6400
after Rs 20 fare (denied): 6400
after rogue 'balance -= 9000': -2600  <-- NEGATIVE

== object-oriented (private balance) ==
deduct 3600: true -> 6400
deduct 2000: false -> 6400  (denied, rule kept)
recharge -500: false -> 6400  (rejected)

== public data member ==
BadCard balance after two rogue deductions: -8000  <-- NEGATIVE
```

7.3 How the class prevents a negative balance

- `balance_` is **private**: code outside the class cannot write `card.balance_ -= 9000;` - the compiler rejects it.
- The only way to reduce it is `deduct(fare)`, and `deduct` first calls `canTravel(fare)`, which requires `balance_ - fare >= MIN_BALANCE_PAISE` (5000). Since 5000 > 0, the balance can never go below Rs 50 after a deduction and therefore never below 0.
- The constructor clamps a negative opening balance to 0 and `recharge()` rejects amounts <= 0 (tested in Task 9).
- A denied journey returns `false` and changes nothing (atomic).

7.4 Which version is easier to change when the minimum balance changes?

In the object-oriented version the minimum balance is the single constant `MIN_BALANCE_PAISE` used by `canTravel`; changing it from Rs 50 to Rs 75 is a **one-line edit** and every caller is automatically correct. In the procedural version

`minBalance` is a global that every function can read and overwrite and, if the value were also copied into messages or other checks, each copy must be found by searching the whole program. The object-oriented version is easier to change and safer, although it is slightly longer to write (a class definition, about 15 lines).

7.5 Alternative concept: public balance - analysis

Solution: `struct BadCard { long long balance = 10000; };` and the gate does `card.balance -= fare;`. **Time:** unchanged, O(1) (a direct subtraction is even one call fewer). **Space:** unchanged, 8 bytes. **What breaks:** the rules R1, R2, R7 and R8 are no longer guaranteed, because *any* part of the program can write the balance. The run above shows two careless deductions on the public member leaving **-8000 paise**, which is exactly the 'negative balance once appeared on a card' defect in the scenario. The class version rejects the same operations. Conclusion: no performance gain, a large loss of safety; the private member with `deduct()` and `canTravel()` is retained.

TASK 8 - Read and Print Data Safely

4 Marks | K5 Evaluate

8.1 Reading with cin and recovering from bad input

When `cin >> hour` is given letters, the extraction fails: `cin.fail()` becomes true, the variable is not changed, and **every further read fails** until the error flag is cleared. The characters that caused the failure stay in the buffer, so the program must (1) call `cin.clear()` to reset the error state and (2) call `cin.ignore(numeric_limits<streamsize>::max(), '\n')` to discard the rest of the line; otherwise the loop would fail forever. Range errors (hour 25, zone 7) are not stream failures, so they are checked separately.

```
template <typename T>
bool readValue(const char* prompt, T& out, int lo, int hi)
{
    while (true) {
        std::cerr << prompt;
        if (std::cin >> out && out >= lo && out <= hi) return true;     // valid and in range
        if (std::cin.eof()) return false;                               // input finished
        if (std::cin.fail()) {                                          // letters typed
            std::cin.clear();                                           // 1. reset error state
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // 2. discard bad line
        }
        std::cerr << "  invalid input, try again\n";
    }
}
```

8.2 Formatting money and the report

Money is stored in paise and printed by integer division and remainder (`f / 100` and `f % 100` with `setw(2)` and `setfill('0')`, as in Task 6) or, in the report, as a double *for display only* with `fixed << setprecision(2)`. `setw(n)` sets the field width of the next item only; `setfill` and `fixed/setprecision` stay in force until changed, so the program restores them (`std::defaultfloat`, `setfill(' ')`). Left alignment (`std::left`) is used for text columns and right alignment for numbers.

8.3 Demonstrated recovery (input file with deliberate errors)

`input/invalid_input.txt` contains: `two` (letters for the journey count), `3`, then for journey 1: `ab` (letters for the hour), `9`, `30`, `7` (unknown zone), `2`, `x` (letters for the category), `1`; and valid data for journeys 2 and 3. The prompts and error messages go to the error stream; the standard output below is the journey report.

Report produced from input/invalid_input.txt (4 rejected values, program recovered every time)

```
No  Time   Zone    Fare      Result    Balance
--------------------------------------------------
1   09:30  Outer   Rs 18.00    OK      Rs 82.00
2   20:00  Middle  Rs 20.00    OK      Rs 62.00
3   10:00  Inner   Rs 10.00    OK      Rs 52.00
```

Report from input/valid_input.txt (4 journeys; the last two are denied by the Rs 50 rule)

```
No  Time   Zone    Fare      Result    Balance
--------------------------------------------------
1   09:30  Outer   Rs 36.00    OK      Rs 64.00
2   20:00  Middle  Rs 10.00    OK      Rs 54.00
3   10:00  Inner   Rs 10.00    DENIED  Rs 54.00
4   18:15  Outer   Rs 18.00    DENIED  Rs 54.00
```

Check by hand (valid input): Outer 09:30 adult is peak: 3000 + 600 = 3600, balance 10000 - 3600 = 6400 = Rs 64.00. Middle 20:00 student is off-peak: 2000, concession 50 % = 1000, balance 5400 = Rs 54.00. Inner 10:00 adult = 1000 would leave 4400 < 5000, so it is **DENIED**; Outer 18:15 student = 3600 / 2 = 1800 would leave 3600, **DENIED**.

8.4 Alternative concept: getline + stoi with try/catch - analysis

Solution: read a whole line with `std::getline`, convert with `std::stoi(line, &used)`, throw if `used != line.size()` (rejects `9x`) or if the value is outside the range; catch `std::exception` and ask again. Run with `input/getline_input.txt` (`abc`, `25`, `9x`, `-5`, `9`, `zz`, `61`, `30`):

```
Hour (0-23):   rejected (stoi), try again
Hour (0-23):   rejected (range), try again
Hour (0-23):   rejected (trailing characters), try again
Hour (0-23):   rejected (range), try again
Hour (0-23): Minute (0-59):   rejected (stoi), try again
Minute (0-59):   rejected (range), try again
Minute (0-59): Accepted 9:30
```

Aspect	cin >> value	getline + stoi + try/catch
Time to process a line of length L	O(L)	O(L): read the line once, convert once
Buffer space	stream buffer only	extra `std::string` of length L (O(L))
Recovery from bad input	must `clear()` and `ignore()`; forgetting either gives an endless loop	no stream error state: the line is simply discarded; the loop is simple
Trailing junk (`9x`)	`cin >> int` accepts 9 and leaves x in the buffer, which breaks the next read	detected with `used != line.size()`
Error information	only true/false	exception type (`invalid_argument` / `out_of_range`)
Cost	cheapest	exception handling is slow, but only when input is invalid

Conclusion: both are O(L). `getline` + `stoi` recovers more simply and catches trailing characters, so it is better for operator typing; `cin >>` with `clear/ignore` is acceptable for the small gate program and is the version demonstrated.

TASK 9 - Implement and Test the Fare Logic

4 Marks | K5 Evaluate

9.1 Implementation

The implementation is `metro_v1.hpp` (fare logic, listed below) with `smartcard.hpp` (Task 7). `task9_tests.cpp` is a table-driven test program.

```
// metro_v1.hpp - original design (3 zones, bool peak, switch on zone)
#pragma once
#include "smartcard.hpp"

enum class Zone { Inner, Middle, Outer };
enum class Category { Adult, Student, Senior };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, 20, 30 (one place only)
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int CONCESSION_PERCENT     = 50;
constexpr int PEAK_AM_START = 8 * 60,  PEAK_AM_END = 10 * 60;   // [08:00, 10:00)
constexpr int PEAK_PM_START = 17 * 60, PEAK_PM_END = 20 * 60;   // [17:00, 20:00)

// half-open intervals: 10:00 and 20:00 are OFF-peak, 08:00 and 17:00 are peak
constexpr bool isPeak(int hour, int minute = 0)
{
    const int m = hour * 60 + minute;
    return (m >= PEAK_AM_START && m < PEAK_AM_END) ||
           (m >= PEAK_PM_START && m < PEAK_PM_END);
}

// integer percentage, rounded half-up: no floating point, no lost paisa
constexpr long long percentOf(long long amount, int percent)
{
    return (amount * percent + 50) / 100;
}

inline bool zoneFromInt(int n, Zone& z)
{
    switch (n) {
        case 0: z = Zone::Inner;  return true;
        case 1: z = Zone::Middle; return true;
        case 2: z = Zone::Outer;  return true;
        default: return false;                 // unknown zone rejected
    }
}
```

```
inline long long fare(Zone z, bool peak, Category c = Category::Adult)
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    long long f = peak ? base + percentOf(base, PEAK_SURCHARGE_PERCENT) : base;
    if (c != Category::Adult) f -= percentOf(f, CONCESSION_PERCENT);
    return f;
}
```

9.2 Test design

Four groups: **normal** (each zone, peak and off-peak, plus both concessions), **boundary** (07:59, 08:00, 09:59, 10:00, 17:00, 19:59, 20:00; balance leaving exactly Rs 50 and one paisa less), **invalid** (unknown zone 7 and -1, negative and zero recharge, negative opening balance, letters typed for the hour) and a **five-journey hand calculation**. Expected values were worked out by hand *before* running.

9.3 Hand calculation of five journeys (card starts at Rs 200.00)

No.	Journey	Working	Fare (paise)	Balance after
1	Adult, Inner, 07:59	off-peak (before 08:00): 1000	1000	19000
2	Adult, Middle, 08:00	peak: 2000 + 20 % = 2000 + 400	2400	16600
3	Adult, Outer, 10:00	off-peak (10:00 excluded): 3000	3000	13600
4	Student, Inner, 17:00	peak 1000 + 200 = 1200; 50 % concession = 600	600	13000
5	Senior, Outer, 20:00	off-peak (20:00 excluded): 3000; 50 % = 1500	1500	11500
	Total	1000 + 2400 + 3000 + 600 + 1500	**8500**	20000 - 8500 = 11500

9.4 Test table - expected versus actual (28 of 28 pass)

```
Case                                          Exp     Act   Result
-----------------------------------------------------------------
Inner off-peak 12:00 adult                   1000    1000   PASS
Middle off-peak 12:00 adult                  2000    2000   PASS
Outer off-peak 12:00 adult                   3000    3000   PASS
Inner peak 09:00 adult                       1200    1200   PASS
Middle peak 18:00 adult                      2400    2400   PASS
Outer peak 18:00 adult                       3600    3600   PASS
Boundary 07:59 Inner (off-peak)              1000    1000   PASS
Boundary 08:00 Inner (peak)                  1200    1200   PASS
Boundary 09:59 Inner (peak)                  1200    1200   PASS
Boundary 10:00 Inner (off-peak)              1000    1000   PASS
Boundary 17:00 Inner (peak)                  1200    1200   PASS
Boundary 19:59 Inner (peak)                  1200    1200   PASS
Boundary 20:00 Inner (off-peak)              1000    1000   PASS
Student Outer off-peak                       1500    1500   PASS
Senior Middle peak                           1200    1200   PASS
Student Inner peak                            600     600   PASS
Balance 6000, fare 1000 -> leaves 5000 (allowed)       1       1   PASS
Balance 5999, fare 1000 -> denied                0       0   PASS
Denied journey keeps balance 6000             6000    6000   PASS
Unknown zone 7 rejected                          0       0   PASS
Negative zone -1 rejected                        0       0   PASS
Recharge -500 rejected                           0       0   PASS
Balance unchanged after bad recharge         10000   10000   PASS
Recharge 0 rejected                              0       0   PASS
Negative opening balance clamped to 0            0       0   PASS
Letters typed for hour -> exception caught       0       0   PASS

--- five journeys, hand total ---
fare 1000 paise, balance 19000
fare 2400 paise, balance 16600
fare 3000 paise, balance 13600
fare 600 paise, balance 13000
fare 1500 paise, balance 11500
Five-journey total (hand: 1000+2400+3000+600+1500)    8500    8500   PASS
Five-journey closing balance (20000-8500)   11500   11500   PASS

28 / 28 tests passed
```

9.5 Alternative concept: table of test cases in a loop - analysis

Solution (the approach used above): an array of `{name, zone, hour, minute, category, expected}` structures

processed by one `for` loop. The alternative is a separate hand-written check per case (`if (fare(...) != 1200) cout << "FAIL";`). For t tests the table needs **O(t) time** (one fare call per row) and **O(t) space** (about 40 bytes per row, 16 rows = under 1 KB). Hand-written checks have the same O(t) time but O(t) *lines of code*. Adding a boundary case to the table costs **one line** (for example `{"Boundary 12:00", 0, 12, 0, 0, 1000}`) and the printing, counting and pass/fail logic is not repeated, so a table is easier to extend and review.

TASK 10 - Compare Expectations with Execution

4 Marks | K5 Evaluate

10.1 Expectation

Before running `task10_totals.cpp` (fixed pseudo-random mix of 20,000 / 500,000 / 3,000,000 journeys using the real `fare()`), I expected: (1) `long long` paise exact; (2) the `double` rupee total to **drift** from the exact total because money in binary floating point is inexact; (3) `float` to lose precision at the large total; (4) a 32-bit `int` to overflow at the network total, because 3,000,000 journeys x about Rs 14 = 4.3 billion paise > 2,147,483,647.

10.2 Measured totals

```
 journeys    paise (LL)      double Rs   dbl err      float Rs    flt err       int32  1st ovfl
    20000      28746300      287463.00  0.00e+00     287463.00       0.00    28746300     never
   500000     720415700     7204157.00  0.00e+00    7204157.00       0.00   720415700     never
  3000000    4320561800    43205618.00  0.00e+00   43140624.00  -64994.00    25594504   1491406
largest running double error seen: 0.00e+00 / 0.00e+00 / 0.00e+00 Rs

long long (paise)        0.848 ns per addition
double (rupees)          1.452 ns per addition
32-bit int (wraps)       0.366 ns per addition
sizeof: long long=8 double=8 int=4
```

Journeys	Exact total (paise)	= Rs	double Rs	float Rs	32-bit int
20,000	28,746,300	287,463.00	287,463.00 (error 0)	287,463.00	28,746,300 (fits)
500,000	720,415,700	7,204,157.00	7,204,157.00 (error 0)	7,204,157.00	720,415,700 (fits)

Journeys	Exact total (paise)	= Rs	double Rs	float Rs	32-bit int
3,000,000	4,320,561,800	43,205,618.00	43,205,618.00 (error 0)	43,140,624.00 (**-64,994 Rs**)	25,594,504 (**wrapped**, overflow at journey 1,491,406)

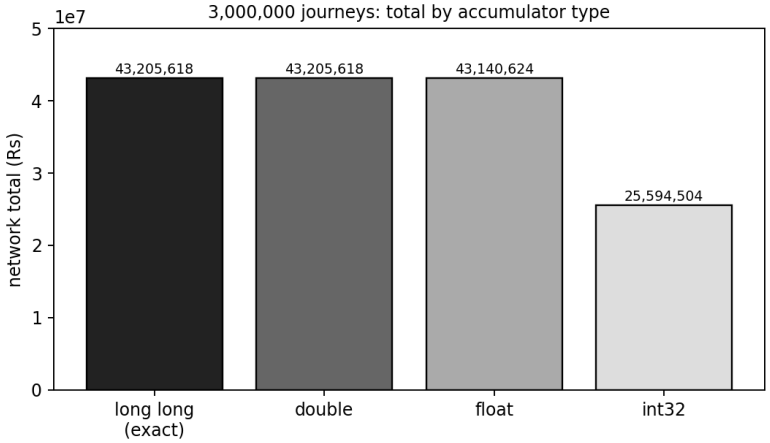


Figure 4 - network total by accumulator type (measured).

10.3 Do double and integer totals agree? Does int overflow?

double: the difference from the exact total was **0 at all three sizes** (also 0 as the largest error at any point during the run). This did **not** match my expectation, and the reason is the tariff: every fare produced by this system is a whole number of rupees (10, 12, 20, 24, 30, 36, 5, 6, 10, 12, 15, 18), and whole numbers are represented exactly by `double` as long as the sum stays below 2^53. Rs 0.10 is different - Task 3 shows an error of 1.6e-4 Rs after 10^7 additions of Rs 0.10. So `double` was correct here **by coincidence of the data, not by design**; any fractional fare would expose the drift. **float:** exact up to 2^24 = 16,777,216, so the 20,000 and 500,000 totals are exact, but at 43 million the spacing between representable values is 4, and fares like 6 or 10 are rounded at each addition: the final total is **Rs 64,994 too low**. **int:** the exact total 4,320,561,800 exceeds INT_MAX (2,147,483,647); the 32-bit accumulator wraps modulo 2^32 to **25,594,504**, and the running total passed INT_MAX at journey **1,491,406**. The totals for 20,000 and

500,000 journeys fit in `int`, so the bug would stay hidden until the network was scaled up - exactly the scenario's growth path.

10.4 Compiler warnings with -Wall -Wextra

Program	Warning / error	Meaning and fix
`task4_predict.cpp`	`warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]`	`s` is converted to unsigned, so -1 becomes 4294967295 and `s < u` is false. Fixed with `static_cast<int>(u)` (task4_fixed.cpp).
`narrow_demo.cpp` (`int x{3.7}; bool f{5};`)	`error: narrowing conversion of '3.7' from 'double' to 'int' [-Wnarrowing]`, same for 5 to `bool`	Brace initialisation refuses conversions that lose information. Fixed with an explicit `static_cast`.
`task12_old_design_demo.cpp`	`warning: enumeration value 'Airport' not handled in switch [-Wswitch]`	A `switch` over an `enum class` did not cover the new zone; the fare fell through as 0 paise. Fixed by the table indexed by zone and `static_assert` (Task 12).
all other programs (tasks 3, 5-12)	none	Compiled with `g++ -std=c++17 -Wall -Wextra -O2` without warnings; no implicit narrowing remains because mixed conversions are written with `static_cast`.

10.5 Reasons for the differences

- Expected drift in `double`, observed none: the data are whole rupees (explained above). The risk is real but data-dependent.
- Expected float failure: observed at the network scale only, because 24 bits give 7 decimal digits.
- Expected int overflow: observed; the largest day (3,000,000) needs 33 bits (4.32e9 > 2^32/2).

10.6 Alternative concept: int, long long or double accumulation - analysis

Accumulator	What happens at 3,000,000 journeys	Failure kind	Time per addition (measured, -O2)	Space
`int` (32-bit)	wraps to 25,594,504 (should be 4,320,561,800)	**overflow** (value does not fit; first at journey 1,491,406)	0.37 ns (vectorised, fastest)	4 B
`long long` (paise)	4,320,561,800 exactly	none (limit 9.22e18)	0.85 ns	8 B
`double` (rupees)	43,205,618.00, exact for this data	**loss of precision** possible for fractional amounts (Task 3)	1.45 ns	8 B
`float`	43,140,624 (64,994 too low)	**loss of precision** (24-bit mantissa)	-	4 B

Conclusion: all additions are O(1) and the time differences are about 1 ns, so they do not matter; `int` fails by overflow, `float`/`double` by loss of precision, and only `long long` paise is exact for any tariff. (Timings are single measurements on one machine and vary run to run.)

TASK 11 - Recommend and Defend the Final Design

4 Marks | K5 Evaluate

11.1 Recommended design

Element	Recommendation	Evidence from my experiments
Money type	`long long` paise everywhere; `double` only to print	Task 3: double drift 1.6e-4 Rs after 10^7 additions of Rs 0.10; paise error is 0. Task 10: int32 wrapped (25,594,504 instead of 4,320,561,800), float lost Rs 64,994.
Constants	One `constexpr` block: `BASE_PAISE[]`, `PEAK_SURCHARGE_PERCENT`, `CONCESSION_PERCENT`, `MIN_BALANCE_PAISE`, peak window in minutes	Task 2; the same constants served Task 6, 9 and 11 unchanged.
Enumerations	`enum class Zone`, `Category` (v2: `Tariff`)	Task 4/12: `-Wswitch` caught a missing zone; narrowing errors caught at compile time.
Classes	`SmartCard` (private balance, `deduct`, `canTravel`, `recharge`); fare logic as constexpr/inline functions; `Pass` added in v2	Task 7: public member gave -8000 paise; class refused. Task 9: 28/28 tests pass, including the Rs 50 boundary and 5999 paise denial.

Peak rule	Half-open intervals in minutes since midnight	Task 6/9: 10:00 and 20:00 consistently off-peak; 08:00 and 17:00 peak.
Input	`cin` with `clear()`/`ignore()` (or `getline` + `stoi` for operator terminals)	Task 8: 4 invalid values recovered; no endless loop.

11.2 How a fare revision is carried out

(1) Edit the number in the one constant block (for example `BASE_PAISE[2] = 3500;`). (2) Recompile with `-Wall -Wextra`. (3) Run the test table; the expected values of the affected rows are updated by hand from the new tariff *before* running, so the tests are independent of the code. (4) Deploy the new build to the gates. No function body is touched.

11.3 Limitations

- **Assumptions not in the brief:** concession is 50 %, applied after the peak surcharge; rounding is half-up in paise; peak is decided by the tap-in time.
- **Recompile and redeploy** are needed for every revision (see 11.5).
- **Concurrency:** if two gates update the same card at once, a local `SmartCard` object does not protect against a race. A network needs a central, atomic ledger (not modelled).
- **Recharge overflow:** `balance_ += paise` has no upper limit check; a maximum card balance should be added.
- **No persistence or time source:** the program trusts the gate clock and does not store the card between runs.
- **Gate processing is single-threaded** in the program; the numbers below are per-core.

11.4 Does the design remain correct for the whole network?

Correctness: all money is `long long`; the largest total measured is 4,320,561,800 paise per day, about 1/2.1 billion of the `long long` limit, so overflow is not a concern for the network (or for years of accumulation). **Speed:** measured with `task11_bench.cpp`, 3,000,000 fare computations took the time shown below (about 13 ns each including the random-number generation), i.e. well under 0.1 s of CPU for a whole day's journeys, while 3,000,000 journeys/day is only about 35 journeys per second on average. **Memory:** `sizeof(SmartCard)` = 8 bytes, so one million cards need 8 MB. Time per journey and memory per card are O(1), so the design scales from one station to the whole network; the limiting factors are the central ledger and communication, not the fare arithmetic.

```
3,000,000 fare computations: 38.107 ms total, 12.7023 ns each (total 4321663800 paise)
Reading 4 fare values from a config file at start-up: 73.952 microseconds (read 1000 2000 3000 6000)
sizeof(SmartCard)=8  sizeof(v2::Pass)=12  sizeof(v2::BASE_PAISE)=32 bytes  sizeof(z[4])=32 bytes
```

11.5 Alternative concept: fare table read from a configuration file - analysis

Solution: at start-up the gate opens `fares.cfg` and reads z values into `std::vector<long long>` (or `long long FARE[z]`). **Extra start-up time:** O(z) - measured about 74 microseconds to read 4 values (a single cold read; it varies). **Space:** z x 8 bytes (32 bytes for 4 zones). **Benefit:** a fare revision needs only a new file, with no recompile and no new build to certify and deploy to every gate. **Costs:** the constants are no longer compile-time (no `constexpr`, no `static_assert` on the table size); the file can be missing, malformed or edited wrongly (a fare of 0 or a negative fare), so it must be validated at start-up (`z` values, all > 0, count equals the number of zones) and the gate must refuse to start otherwise; and the file itself must be protected. Recompiling and redeploying gives a tested, immutable tariff but takes hours for a whole network; reading a file takes microseconds but moves the risk to the data.

Final decision: keep the **compiled constexpr table** for this coursework design because it is checked by the compiler and by the tests; for the full network, move only the **numbers** to a validated configuration file while keeping the same `SmartCard`, enums and `fare()` logic.

TASK 12 - Day-4 Design Challenge: Re-evaluate and Adapt the Design

8 Marks | K5 Evaluate

Changed requirement: fourth zone Airport (Rs 60), 10 % weekend discount that does **not** combine with the peak surcharge, and a monthly pass with unlimited journeys in two chosen zones. **Assumptions:** weekend = Saturday and Sunday; weekend discount applies to the base fare and replaces the peak surcharge; the concession (50 %) still applies afterwards; a pass covers a journey whose farthest zone is one of its two chosen zones, in its month; a pass-covered journey deducts nothing and does not need the Rs 50 minimum.

A. Impact analysis (2 marks)

Part of v1 design	Affected?	Why
`BASE_PAISE[]`	**Yes**	Needs a fourth entry `6000` (Airport).
`enum class Zone`	**Yes**	Add `Airport`.
`switch` on zone in `fare()`	**Yes**	A new `case`; if forgotten the Airport journey is free (-Wswitch warns).
`bool peak` parameter	**Yes**	Two kinds of price change (surcharge, discount) that exclude each other: a bool cannot express three states Standard / Peak / Weekend; becomes `enum class Tariff`.
`isPeak()` expression	No	Peak window unchanged; but it must be bypassed on weekends (`tariffFor`).
New constant	**Yes (new)**	`WEEKEND_DISCOUNT_PERCENT = 10`.
`percentOf()`, concession logic	No	Reused as is.
`SmartCard`	No	Balance rules (R1, R2, R7, R8) are unaffected; a pass journey simply deducts nothing.
Pass	**Yes (new class)**	New data (two zones, month) and behaviour (`covers`); does not belong inside `SmartCard`.
Gate logic	**Yes**	New `travel()` function: check pass first, then card.

B. Evaluation of the original design (2 marks)

`task12_old_design_demo.cpp` applies the *original* switch design to the new requirements:

```
Airport (should be 6000) computed by old code: 0 paise
Saturday 09:00 Inner, naive patch: 1080 paise (required 900)
```

Worked example	Original design result	Required	Verdict
Airport journey, off-peak, adult	Adding `Airport` to the enum without updating the switch: **0 paise** (compiler: `-Wswitch` warning 'enumeration value Airport not handled'); BASE_PAISE has only 3 entries so an index of 3 would read past the array	6000 paise	Switch **cannot absorb** the new zone safely
Saturday 09:00, Inner, adult	Patching 'weekend 10 %' after the peak surcharge gives 1200 - 120 = **1080**	1000 - 10 % = **900**	`bool peak` **cannot absorb** exclusive discount rules
Monthly pass, Middle journey, balance Rs 40	`SmartCard::canTravel(0)` is false because 4000 - 0 < 5000, so a pass holder would be refused	allowed, no deduction	Pass must be checked **before** the card; pass logic does not fit inside `canTravel`
Rs 50 rule, negative balance rule, recharge rule	`SmartCard` unchanged, still correct (all Task 9 card tests still pass)	unchanged	`SmartCard` **can absorb** the change as is

Evaluation: the switch-based fare logic and the `bool peak` are too rigid (modify), the `SmartCard` class is sound (retain), and the pass needs a new class (add). A complete redesign is not justified, because the rules that caused the earlier defects (exact paise, private balance, one constant block) are all still valid.

C. Adaptation: modified design and code (2 marks)

Decision per component: **RETAIN** `SmartCard` and `percentOf`; **MODIFY** zones, fare table (now indexed by zone, size guarded by `static_assert`), tariff (bool becomes `enum class Tariff`), and the discount rule (weekend wins over peak); **ADD** `Pass` and the gate function `travel()`.

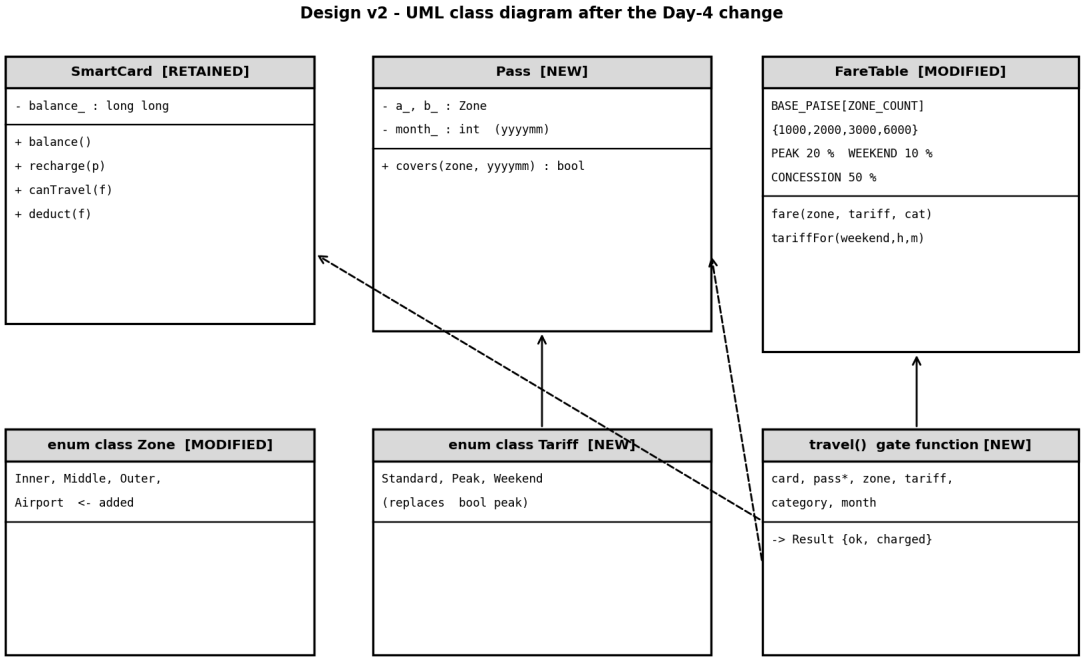


Figure 5 - UML class diagram of the adapted design (v2).

```
// metro_v2.hpp - adapted design: 4 zones, weekend discount, monthly pass
// SmartCard (smartcard.hpp) is RETAINED unchanged; fare logic MODIFIED; Pass class ADDED.
#pragma once
#include <cstddef>
#include "smartcard.hpp"

namespace v2 {

enum class Zone     { Inner, Middle, Outer, Airport };      // new value appended at the end
enum class Category { Adult, Student, Senior };
enum class Tariff   { Standard, Peak, Weekend };            // replaces bool peak: 3 mutually exclusive states

constexpr std::size_t ZONE_COUNT = 4;
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000, 6000};  // Airport = Rs 60
static_assert(static_cast<std::size_t>(Zone::Airport) + 1 == ZONE_COUNT, "fare table must cover every zone");

constexpr int PEAK_SURCHARGE_PERCENT   = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr int CONCESSION_PERCENT       = 50;

constexpr bool isPeakTime(int hour, int minute = 0)
{
    const int m = hour * 60 + minute;
    return (m >= 8 * 60 && m < 10 * 60) || (m >= 17 * 60 && m < 20 * 60);
}
constexpr long long percentOf(long long amount, int percent) { return (amount * percent + 50) / 100; }

// weekend wins: the 10% discount and the 20% surcharge are never combined
constexpr Tariff tariffFor(bool weekend, int hour, int minute = 0)
{
    return weekend ? Tariff::Weekend : (isPeakTime(hour, minute) ? Tariff::Peak : Tariff::Standard);
}

inline long long fare(Zone z, Tariff t, Category c = Category::Adult)
{
    const long long base = BASE_PAISE[static_cast<std::size_t>(z)];       // table indexed by zone
    long long f = base;
    switch (t) {
        case Tariff::Standard: break;
        case Tariff::Peak:     f = base + percentOf(base, PEAK_SURCHARGE_PERCENT); break;
        case Tariff::Weekend:  f = base - percentOf(base, WEEKEND_DISCOUNT_PERCENT); break;
    }
    if (c != Category::Adult) f -= percentOf(f, CONCESSION_PERCENT);
    return f;
}
```

```
// Monthly pass: unlimited journeys whose farthest zone is one of two chosen zones, in one month
class Pass {
public:
    Pass(Zone a, Zone b, int yyyymm) : a_{a}, b_{b}, month_{yyyymm} {}
    bool covers(Zone z, int yyyymm) const { return yyyymm == month_ && (z == a_ || z == b_); }
private:
    Zone a_, b_;
    int  month_;
};

struct Result { bool ok; long long charged; };

inline Result travel(SmartCard& card, const Pass* pass, Zone z, Tariff t, Category c, int yyyymm)
{
    if (pass && pass->covers(z, yyyymm)) return {true, 0};   // covered journey: nothing deducted
    const long long f = fare(z, t, c);
    return {card.deduct(f), f};
}

} // namespace v2
```

D. Evidence-based defence (2 marks)

`task12_tests.cpp` runs 25 cases against the adapted design (all with expected values worked out by hand first):

```
Case                                                    Exp    Act   Result
-------------------------------------------------------------------------------
Airport standard adult                                 6000   6000   PASS
Airport peak adult (6000 + 20%)                        7200   7200   PASS
Airport weekend adult (6000 - 10%)                     5400   5400   PASS
Airport student standard                               3000   3000   PASS
Inner standard (regression)                            1000   1000   PASS
Middle peak (regression)                               2400   2400   PASS
Outer peak senior (regression)                         1800   1800   PASS
Saturday 09:00 Inner: weekend only                      900    900   PASS
Saturday 18:00 Outer: weekend only                     2700   2700   PASS
Weekday 09:00 Inner: peak only                         1200   1200   PASS
Weekday 10:00 Inner: standard                          1000   1000   PASS
Weekday 20:00 Inner: standard                          1000   1000   PASS
Weekend student Middle (2000-10%=1800, /2)              900    900   PASS
Pass covers Middle: charged 0                             0      0   PASS
Pass covers Middle: balance unchanged                  6000   6000   PASS
Pass covers Inner: journey allowed                        1      1   PASS
Outer not on pass: normal fare charged                 3000   3000   PASS
Outer not on pass: balance reduced                     7000   7000   PASS
Expired pass (next month): fare charged                1000   1000   PASS
Pass journey allowed even with balance Rs 40              1      1   PASS
Pass journey leaves balance unchanged                  4000   4000   PASS
No pass, Airport, balance 6000: denied                    0      0   PASS
Denied journey keeps balance                           6000   6000   PASS
Airport leaves exactly Rs 50: allowed                     1      1   PASS
Closing balance exactly 5000                           5000   5000   PASS

25 / 25 tests passed
```

Requirement	Test evidence
Airport zone Rs 60	6000 standard; 7200 peak (6000 + 20 %); 5400 weekend (6000 - 10 %); 3000 student.
Old zones unaffected (regression)	Inner 1000, Middle peak 2400, Outer peak senior 1800 - same as Task 9.
Weekend 10 % never combined with peak	Saturday 09:00 Inner = 900 (not 1080); Saturday 18:00 Outer = 2700; weekday 09:00 = 1200; weekday 10:00 and 20:00 = 1000 (boundaries unchanged).
Monthly pass	Covered zone: charged 0, balance unchanged (even at Rs 40); uncovered zone charged normally (3000); expired month charged (1000).
Rs 50 rule still holds	Airport with balance 11000 is allowed leaving exactly 5000; balance 6000 is denied and stays 6000.

Trade-offs and decision. *Retain/modify/add* instead of a full redesign: three of the four old components needed at most a one-line change, `smartcard.hpp` is **byte-for-byte unchanged** (so the old card tests remain valid), and 25/25 tests pass. A full polymorphic redesign (`FareRule` base class with `PeakRule`, `WeekendRule`, `PassRule`) would make future rules easier to add but costs a virtual call per rule per journey and more code for four rules; it is the right next step only if the tariff keeps growing. Keeping `Pass` outside `SmartCard` keeps the card simple (single responsibility) at the price of a small coordinating function, `travel()`. Time per journey stays O(1); memory is 32 bytes for the fare table, 12 bytes per `Pass` and 8 bytes per card (`task11_bench`). **Limitations:** the pass semantics (farthest zone in the two chosen zones) and the 50 % concession are assumptions that must be confirmed with the metro authority; a pass is not yet linked to a card identifier; weekend detection (public holidays) is supplied by the caller. **Final decision: MODIFY the original design (retain `SmartCard`, modify the fare logic, add `Pass`).**

Submission files (GitHub repository)

`src/` (all programs and headers), `input/` (valid_input.txt, invalid_input.txt, getline_input.txt), `output/` (program outputs and compiler warnings), `screenshots/` (output captures), `README.md`. Compile every program with `g++ -std=c++17 -Wall -Wextra -O2 file.cpp -o file`.